

Assignment #2

Primary/Backup Key/Value Service

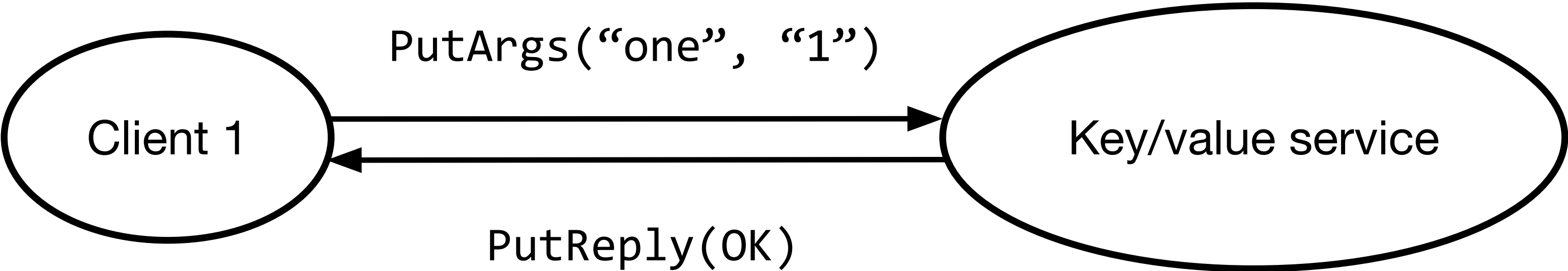
Xijiao Li (xl2950)

Spring 2022

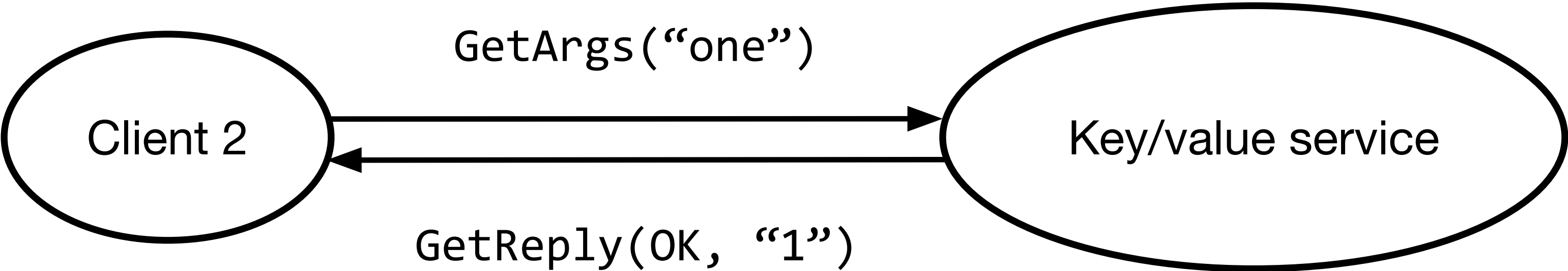
Key/value service

- The service maintains a simple database of key/value pairs. Each key maps to exactly one value.
- Three RPCs:
 - Put, PutHash, Get
- Stateful
 - Stateful services keep track of sessions or transactions and react differently to the same inputs based on the history.
 - The key/value service is stateful: needs to know all the key/value pairs added so far in order to serve future requests.

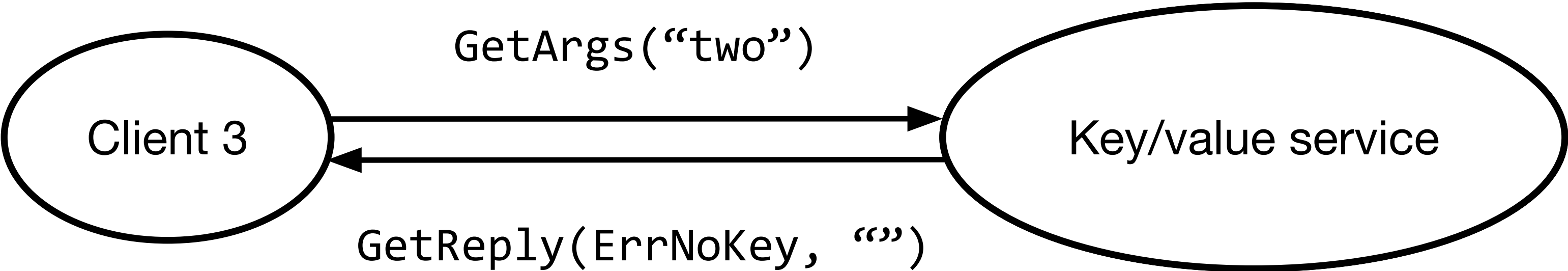
Key/value service



Key/value service



Key/value service

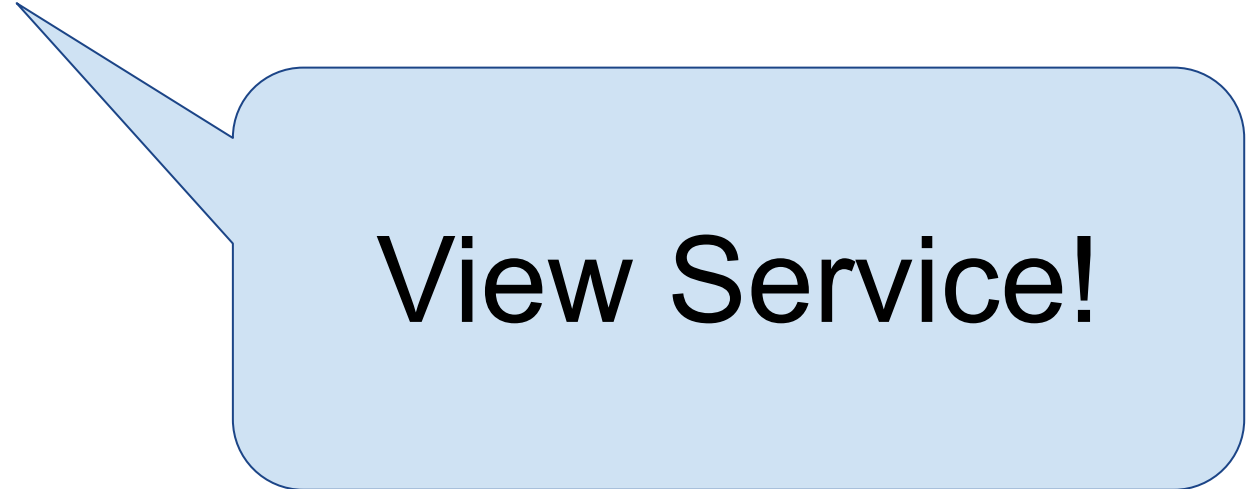


Primary/backup Replication

- Motivation: Fault tolerance
 - we'd like a service that continues despite failures.
- Mechanism: Replicate and separate servers
- Goal #1: Provide a highly reliable service
 - Even if there are some server/network failures, it can continue operation after failure.
- Goal #2: Servers should behave just like a single, more reliable server.

Primary/backup Replication

- Nominate one server the *primary*, one server the *backup*.
 - Clients send all operations (**Get**, **Put**) to current primary.
 - The primary serves clients' operations.
- Should be only one primary at a time.
- Need to keep clients, primary, and backup in sync on who is primary and who is backup.



View Service!

Part A: The Viewservice

- View service decides who is primary, who is backup.
 - A **View** contains three pieces of info: **Viewnum**, **Primary**, **Backup**
- Clients and servers consult view service rather than deciding on their own to make sure they all in agreement.
 - The viewservice monitors whether each known server is alive or dead.
 - A client checks with the viewservice to find the current primary.
 - The servers cooperate with the viewservice to ensure that at most one primary is active at a time.
- For now, assume that view service never fails.

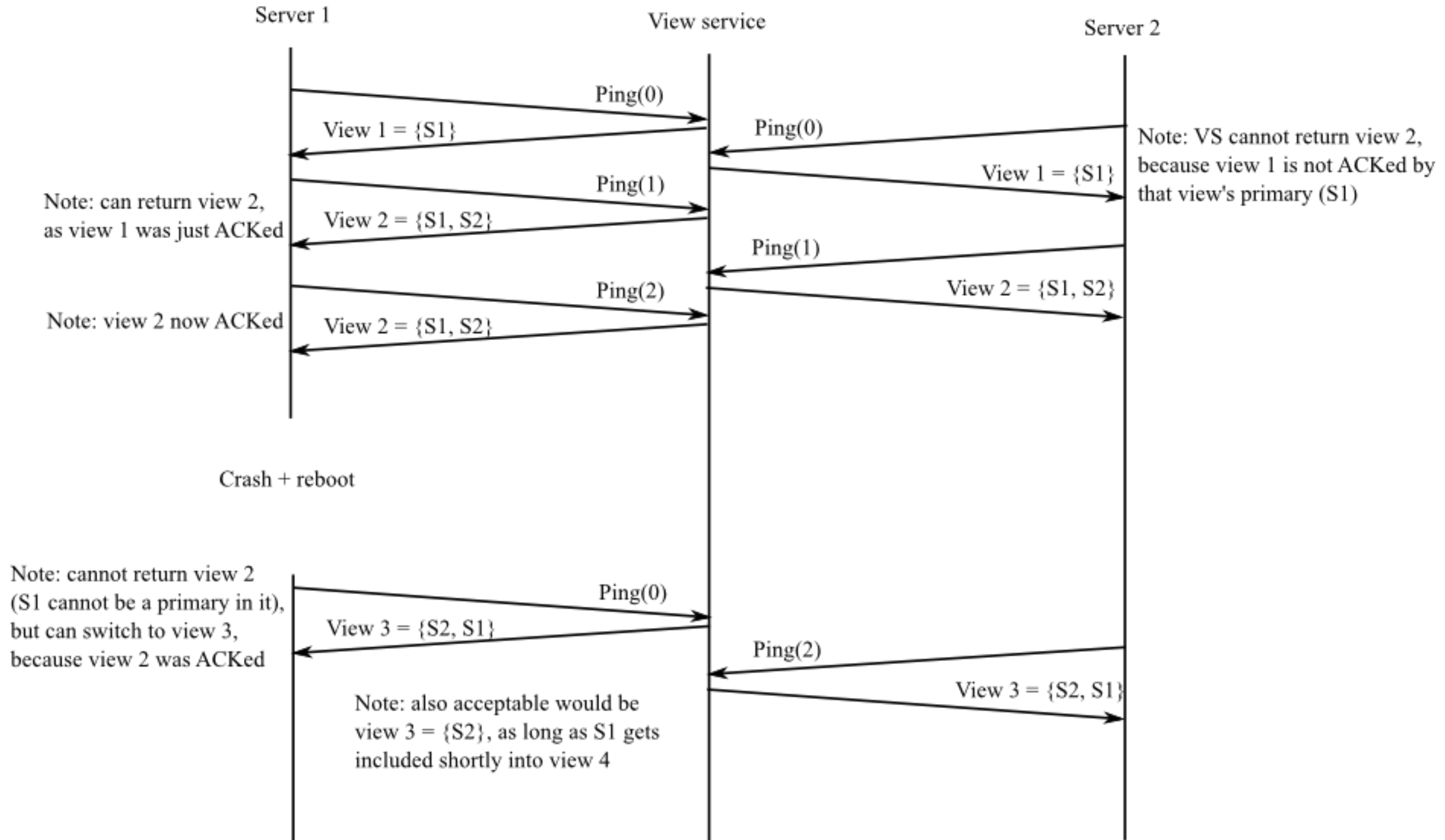
Monitor Server Aliveness

- Each server pings the view server per `PingInterval`.
 - A `Ping` lets the view server know that the key/value server is alive, and informs the view server of the most recent `View` it knows about.
 - The view server replies with a description of the current `View`.
- If the view server doesn't receive a `Ping` from a server for a certain number of intervals, it considers the server to be dead.
- When a server restarts after a crash, it should send one or more `Ping` with `Viewnum(0)` to inform that it has crashed.

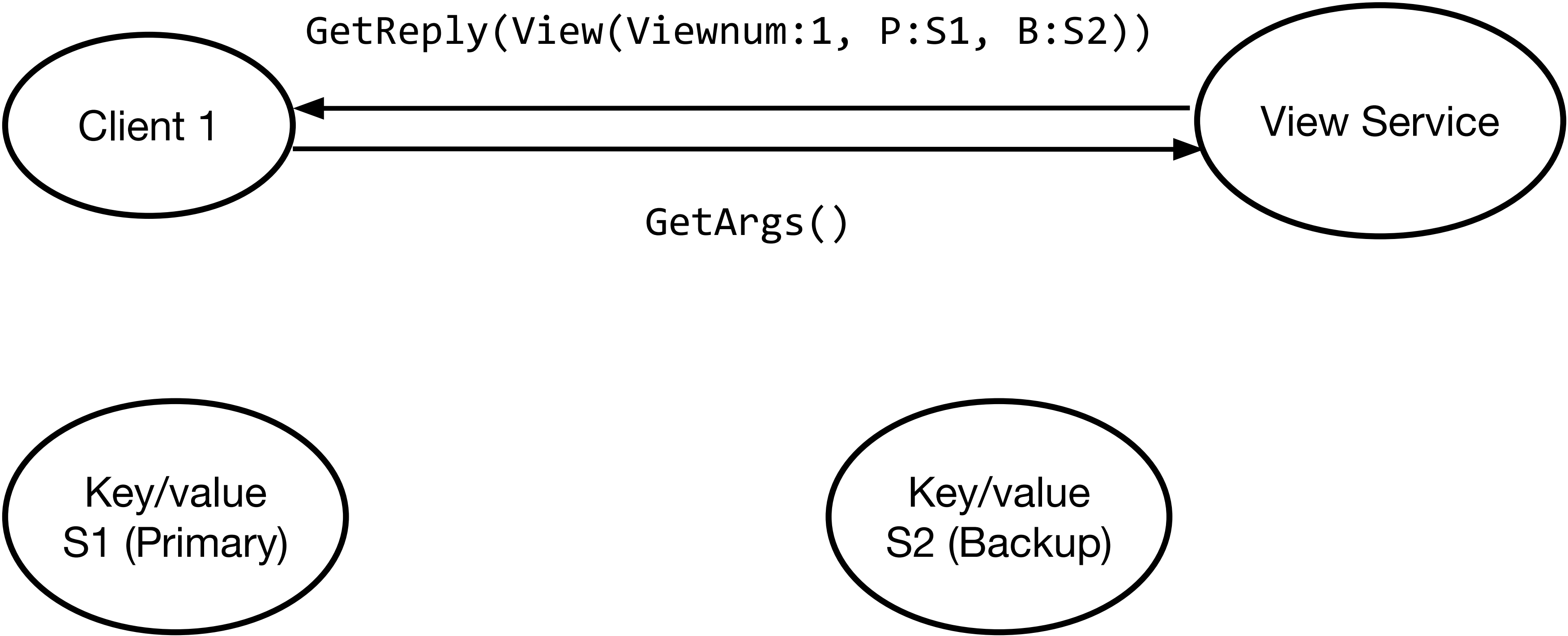
New View

- The viewservice proceeds to a new view when either
 1. it hasn't received a **Ping** from the primary or backup;
 2. the primary or backup crashed and restarted;
 3. there is no backup and there's an idle server (i.e. a server that sent a **Ping** but is neither the primary nor the backup).
- But the view server must NOT change views until the primary from the current view acknowledges that it is operating in the current view (i.e. by sending a **Ping** with the current view number).

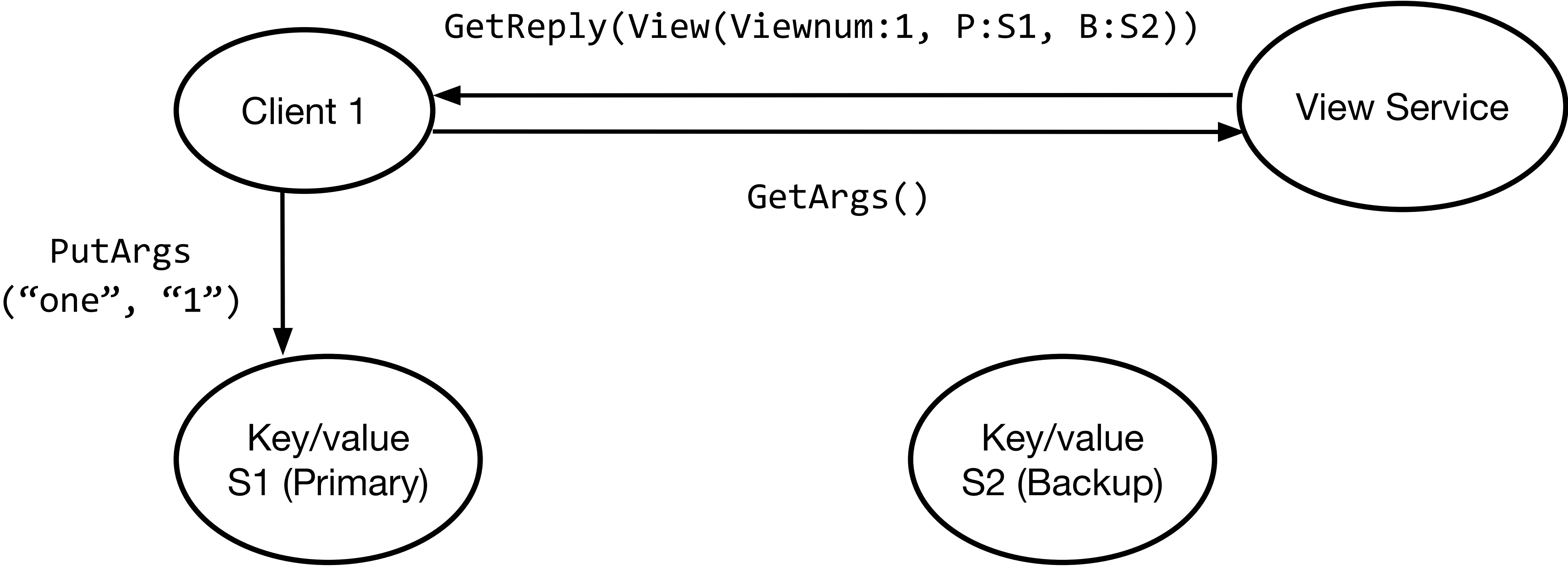
Example Scenario



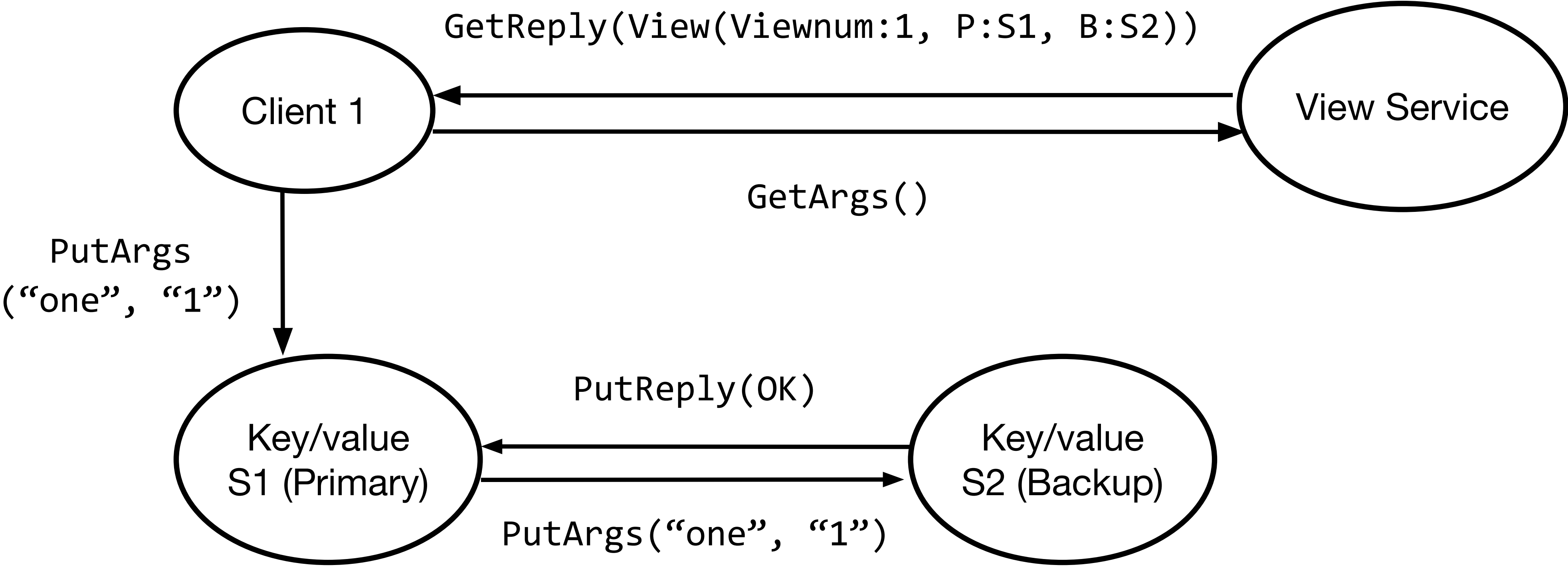
Part B: The Primary/backup Key/value Service



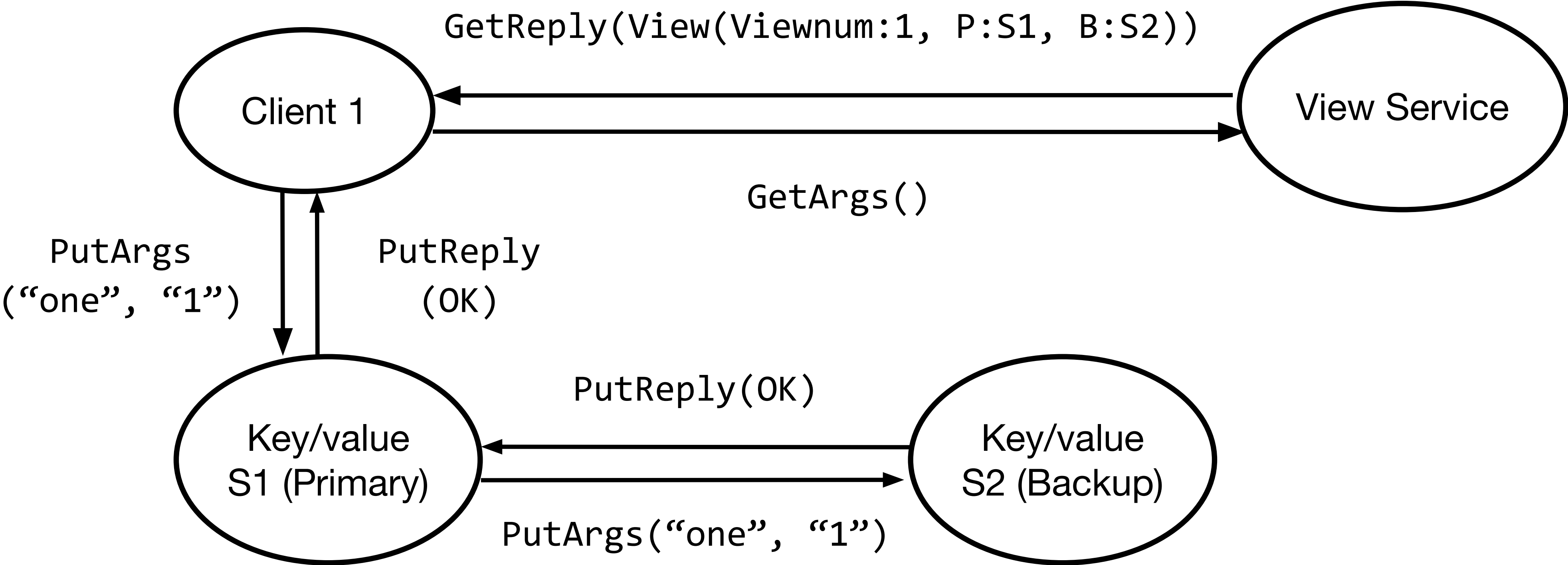
Part B: The Primary/backup Key/value Service



Part B: The Primary/backup Key/value Service



Part B: The Primary/backup Key/value Service



Basic Logistics

1. Primary receives the operation from client.
2. Primary sends operation to backup and waits for ACK.
3. Backup performs operation and acks to primary.
4. Primary performs operation and ACK to the client.

Q&A

- Can a server be alive but considered “dead” by view server?
 - Yes, in the case of network failure or partition.
- Can more than one server think it is the primary at the same time?
 - Yes, in the case of network failure or partition.
- Are there cases when the system can't process new client requests?
 - View server fails;
 - Network fails entirely (hard to get around this one);
 - Client can reach view server but it can't reach primary;
 - Primary fails and there is no backup;
 - Primary fails before completing transfer its states to new backup.